

人脸表情识别实验指南

一、实验概述

1.1 实验目的

- 理解人工智能技术在物联网应用中的集成方法
- 掌握Python开发环境的搭建与依赖管理，培养物联网软件开发能力
- 了解边缘计算中的轻量级模型（BlazeFace）及其在嵌入式场景的应用
- 能够独立完成物联网智能感知应用的开发与部署

1.2 项目功能介绍

本项目实现了一个基于CNN的人脸表情识别系统，是**智能物联网感知层**的典型应用案例，能够识别**8种基本表情**：

编号	英文	中文
0	anger	发怒
1	disgust	厌恶
2	fear	恐惧
3	happy	开心
4	sad	伤心
5	surprised	惊讶
6	neutral	中性
7	contempt	蔑视

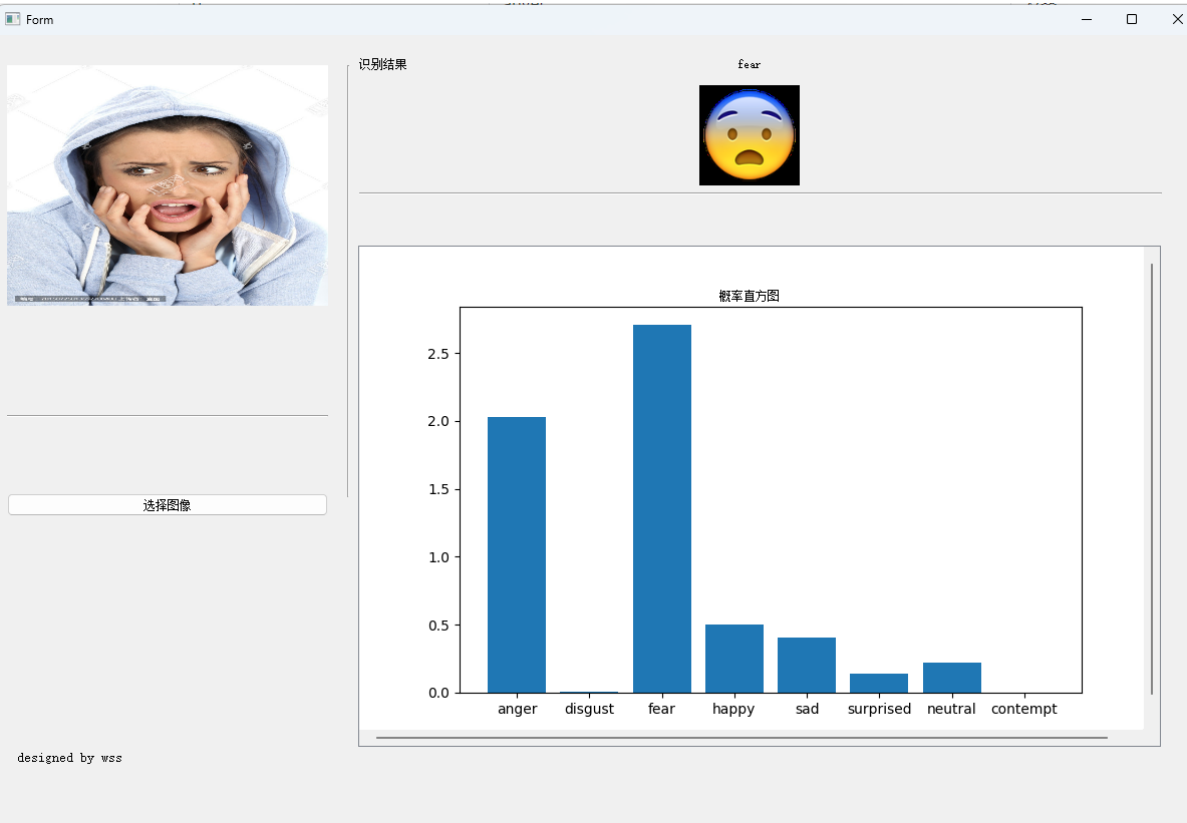
主要功能：

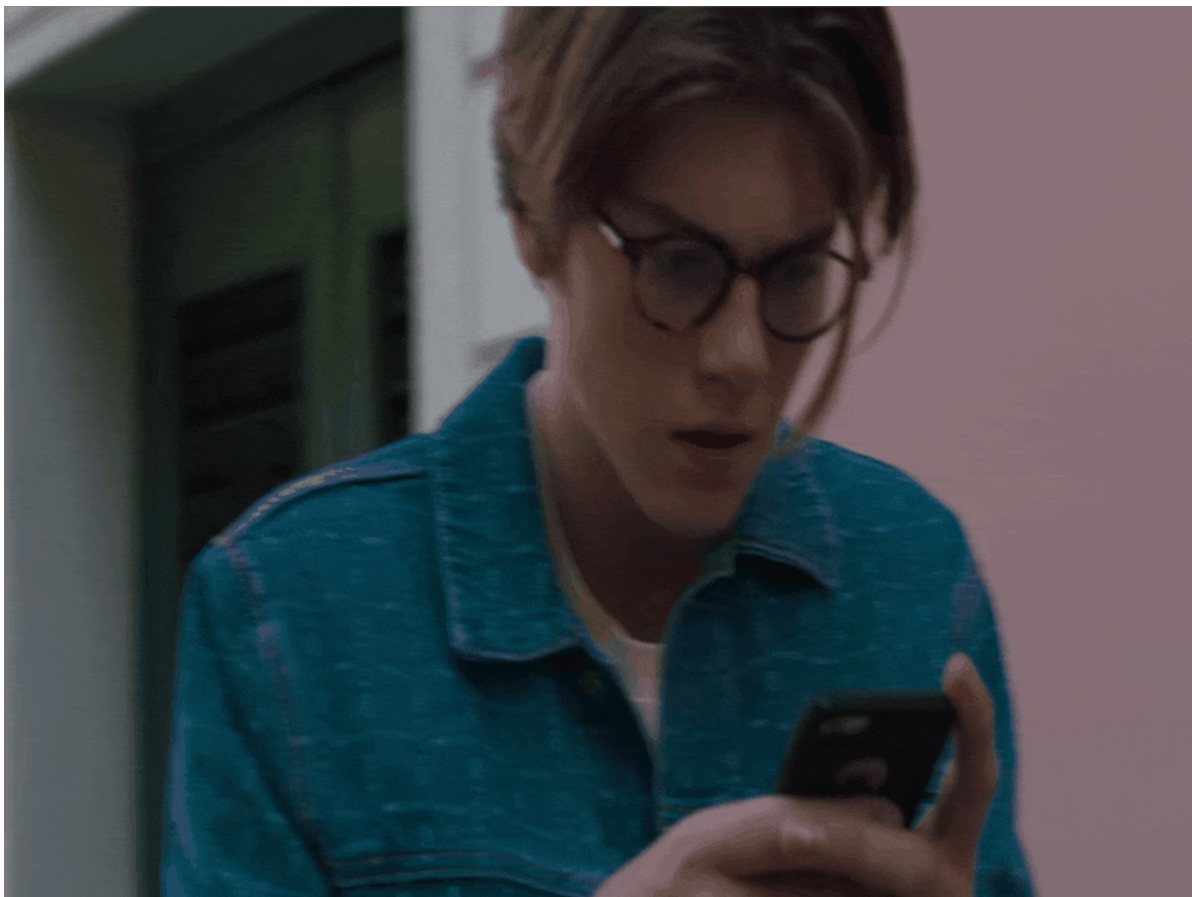
- 图形界面（GUI）图片识别——物联网应用层人机交互
- 摄像头实时表情检测——物联网感知层实时数据采集
- 视频文件表情分析——批量数据处理与离线分析

物联网应用场景：

场景	应用描述
智能家居	根据家庭成员表情自动调节灯光、音乐等环境参数
智慧养老	监测老年人情绪状态，异常时自动通知家属
车载系统	检测驾驶员疲劳或愤怒情绪，发出安全预警
智能客服	识别顾客情绪，优化服务质量
教育物联网	监测学生课堂注意力与情绪，辅助教学评估

最终效果：





1.3 实验环境要求

项目	推荐配置
操作系统	Windows 10/11（主要）、Linux、macOS
Python	3.6 - 3.8
内存	8GB及以上
显卡	NVIDIA显卡（可选，支持GPU加速）
硬盘	5GB可用空间

二、环境搭建

2.1 Python安装

Windows系统

1. 下载 Anaconda：访问 <https://www.anaconda.com/download>
2. 运行安装程序，选择"Just Me"（推荐）
3. 勾选"Add Anaconda to my PATH environment variable"（可选，但方便命令行使用）

验证安装

打开命令提示符（Win+R输入 `cmd`），执行：

```
python --version  
# 应显示 Python 3.6.x 或更高版本
```

2.2 创建虚拟环境

虚拟环境可以隔离项目依赖，避免版本冲突。

```
# 创建名为FER的虚拟环境，Python版本3.6  
conda create -n FER python=3.6 -y  
  
# 激活环境  
conda activate FER
```

注意：后续所有命令都需要在激活的FER环境中执行。每次打开新的终端都需要重新激活环境。

2.3 安装依赖

2.3.1 GPU版本（推荐，如果有NVIDIA显卡）

```
# 安装CUDA工具包和cuDNN  
conda install cudatoolkit=10.1 cudnn=7.6.5 -y  
  
# 安装TensorFlow GPU版本  
pip install tensorflow-gpu==2.3.1
```

2.3.2 CPU版本（无NVIDIA显卡）

```
# 安装TensorFlow CPU版本  
pip install tensorflow==2.3.1
```

2.3.3 安装其他依赖

进入项目目录后执行：

```
# 切换到项目目录（请替换为实际路径）  
cd D:\你的路径\FacialExpressionRecognition  
  
# 安装requirements.txt中的依赖  
pip install -r requirements.txt
```

2.3.4 常见安装问题

问题1: dlib安装失败

dlib需要C++编译环境，Windows下常见解决方法：

方法1: 使用预编译包 (推荐)

```
pip install dlib-binary
```

方法2: 下载预编译whl文件

访问 <https://pypi.org/project/dlib/> 或第三方源下载对应版本的whl文件

然后执行 `pip install dlib-19.6.1-cp36-cp36m-win_amd64.whl`

问题2: numpy版本冲突

如果出现numpy版本问题, 可以指定版本

```
pip install numpy==1.18.5
```

问题3: opencv版本问题

```
pip install opencv-python==3.4.4.19
```

2.4 下载数据集和模型

1. 将 `model.zip` 解压到 `models/` 文件夹, 得到以下文件:

```
models/  
├─ cnn3_best_weights.h5    # 主模型权重  
└─ ...
```

2. 将 `data.zip` 解压到 `dataset/` 文件夹, 得到以下结构:

```
dataset/  
├─ fer2013/  
│   ├─ Training/          # 训练集  
│   ├─ PublicTest/       # 验证集  
│   └─ PrivateTest/      # 测试集  
├─ jaffe/  
└─ ck+/  

```

三、运行实验

3.1 项目目录结构

```
FacialExpressionRecognition/  
├─ src/                    # 源代码  
│   ├─ gui.py              # GUI程序入口  
│   ├─ recognition.py      # 表情识别核心逻辑  
│   ├─ model.py            # CNN模型定义  
│   ├─ train.py            # 训练脚本  
│   ├─ data.py             # 数据加载  
│   ├─ utils.py            # 工具函数  
│   └─ blazeface/          # BlazeFace人脸检测  
├─ models/                 # 模型权重文件  
├─ dataset/                # 数据集  
└─ input/test/             # 测试图片存放位置
```

— output/	# 输出结果
— assets/	# 资源文件（字体等）
— requirements.txt	# 依赖列表

3.2 启动GUI界面

```
# 确保在FER环境中
conda activate FER

# 进入src目录
cd src

# 启动GUI
python gui.py
```

GUI使用步骤：

1. 点击"打开图片"按钮，选择包含人脸的图片
2. 系统自动检测人脸并识别表情
3. 识别结果会显示在界面上，并保存到 `output/rst.png`

3.3 摄像头实时检测

```
# 进入src目录
cd src

# 启动摄像头检测（按ESC键退出）
python recognition_camera.py
```

可选参数：

```
# 使用指定摄像头（默认0）
python recognition_camera.py --source 1

# 处理视频文件
python recognition_camera.py --video_path ../test_video.mp4
```

3.4 命令行测试

```
cd src

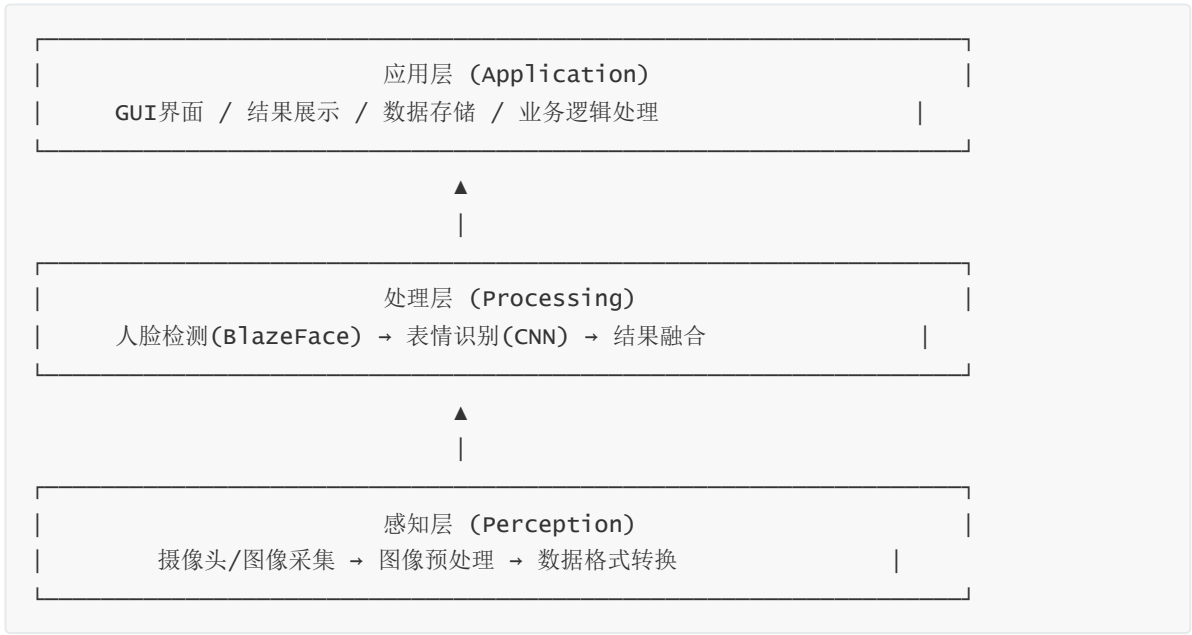
# 直接运行识别脚本
python recognition.py
```

识别结果将保存到 `output/rst.png`。

四、技术原理讲解

4.1 系统整体架构

本系统是一个典型的物联网智能感知应用，采用分层架构设计：



数据流向：



处理流程：

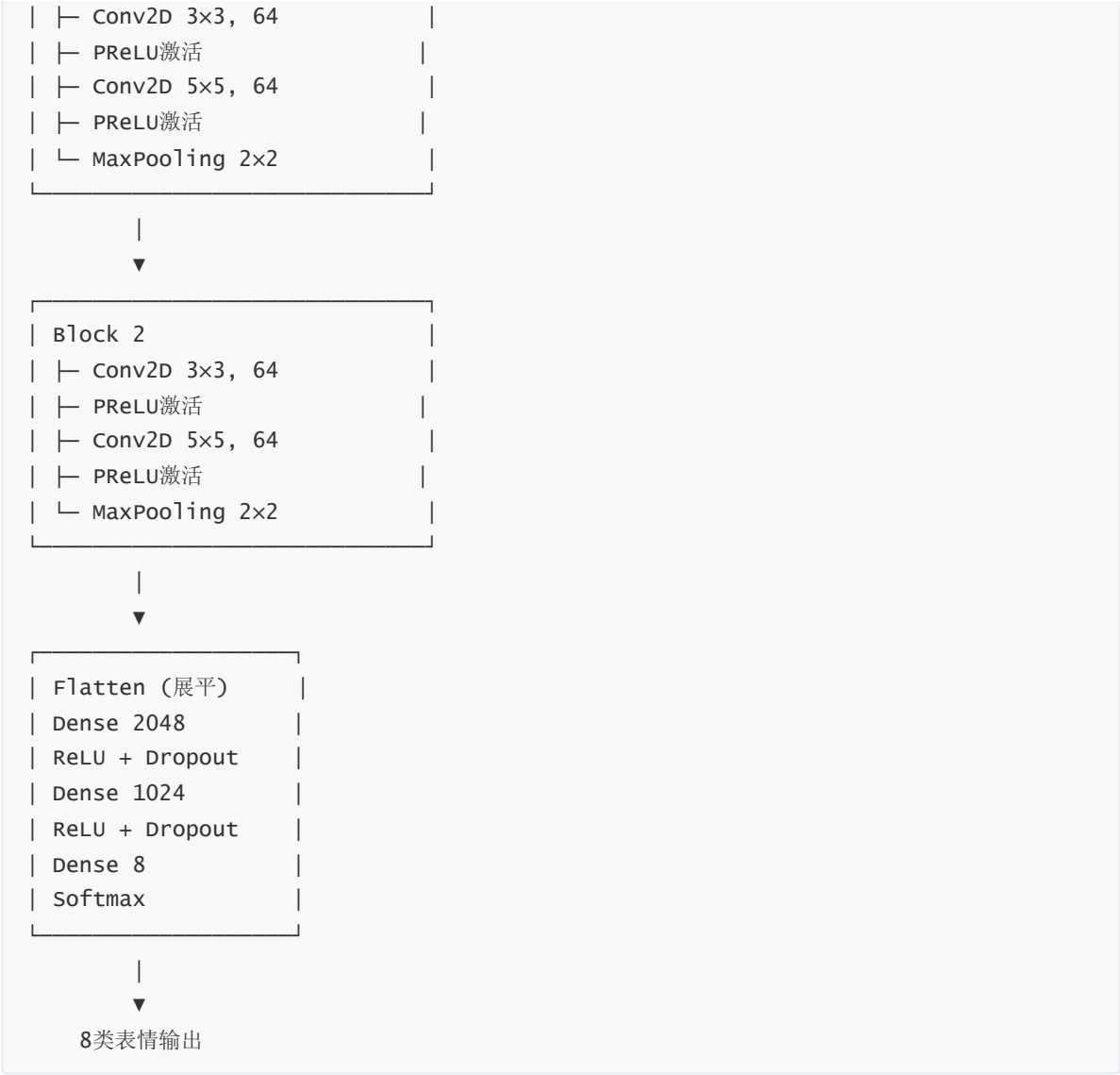
- 人脸检测**：从图片中定位人脸区域
- 人脸预处理**：裁剪、缩放到48x48像素、灰度化
- 数据增广**：对同一人脸生成多个变体（提高识别稳定性）
- 表情分类**：CNN模型预测表情类别
- 结果融合**：对多个增广结果的预测概率求和

4.2 CNN模型架构

本项目使用 **CNN3** 模型，参考论文 "A Compact Deep Learning Model for Robust Facial Expression Recognition".

网络结构图





关键组件解释

组件	作用
卷积层 (Conv2D)	提取图像特征，如边缘、纹理、形状
PReLU激活	引入非线性，可学习的负值斜率，比ReLU更适合表情识别
最大池化 (MaxPooling)	降低特征图尺寸，保留主要特征，减少计算量
Dropout	训练时随机丢弃神经元，防止过拟合
Softmax	将输出转换为概率分布，8个类别概率之和为1

4.3 PReLU激活函数

PReLU (Parametric ReLU) 是ReLU的改进版本：

$$\begin{aligned} f(x) &= x && \text{if } x > 0 \\ &= \alpha \cdot x && \text{if } x \leq 0 \end{aligned}$$

其中 α 是可学习参数，在训练过程中自动调整。

优点：

- 避免ReLU的"神经元死亡"问题
- 对负值区域有非零梯度，训练更稳定
- 特别适合表情识别中微妙的特征差异

4.4 人脸检测：BlazeFace

本项目使用 **BlazeFace** 进行人脸检测，这是Google开发的轻量级人脸检测器。

特点

特性	说明
速度	在移动设备上可达200+ FPS
精度	基于SSD架构，精度高
轻量	模型仅~500KB（TFLite格式）
关键点	可输出6个人脸关键点（眼、鼻、嘴等）

工作原理

1. **输入预处理**：将图片缩放到128×128像素
2. **锚框生成**：在特征图上生成多个候选框
3. **置信度过滤**：根据得分阈值筛选高质量检测
4. **NMS处理**：去除重叠的检测框

4.5 测试时增广 (TTA)

为了提高预测稳定性，系统对每个人脸生成**7种变体**：

```
# 原始图片
resized_images.append(face_img)

# 裁剪变体
resized_images.append(face_img[2:45, :])           # 上部裁剪
resized_images.append(face_img[0:45, 0:45])        # 左上角
resized_images.append(face_img[2:47, 0:45])        # 调整后裁剪
resized_images.append(face_img[2:47, 2:47])        # 中心裁剪

# 水平翻转
resized_images.append(cv2.flip(face_img, 1))
```

最终预测：对7个变体的预测概率求和，取最大值对应的类别。

这种方法可以减少因人脸检测偏差带来的误差，提高识别的鲁棒性。

4.6 数据增强

训练时使用数据增强技术扩充数据集，提高模型泛化能力：

```
ImageDataGenerator(  
    rotation_range=10,      # 随机旋转±10度  
    width_shift_range=0.05, # 水平平移5%  
    height_shift_range=0.05, # 垂直平移5%  
    horizontal_flip=True,   # 水平翻转  
    shear_range=0.2,       # 剪切变换  
    zoom_range=0.2         # 随机缩放  
)
```

五、训练模型（简要介绍）

5.1 训练命令

```
cd src  
  
# 在fer2013数据集上训练300个epoch  
python train.py --dataset fer2013 --epochs 300 --batch_size 32  
  
# 可选数据集: fer2013, jaffe, ck+
```

5.2 训练参数说明

参数	默认值	说明
<code>--dataset</code>	fer2013	训练数据集
<code>--epochs</code>	200	训练轮数
<code>--batch_size</code>	32	每批样本数
<code>--plot_history</code>	True	是否绘制训练曲线

5.3 训练过程

1. **数据加载**: 从 `dataset/` 目录读取图片和标签
2. **数据划分**: 训练集用于学习, 验证集用于调参
3. **模型编译**:
 - 优化器: SGD (学习率0.01, 动量0.9)
 - 损失函数: 交叉熵损失
4. **模型训练**: 迭代更新权重, 保存最佳模型
5. **结果可视化**: 生成损失和准确率曲线图

5.4 训练输出

- 模型权重: `models/cnn3_best_weights.h5`
- 损失曲线: `assets/his_loss.png`
- 准确率曲线: `assets/his_acc.png`

六、实验任务

6.1 基础任务（必做）

1. 环境搭建

- ☐ 成功创建FER虚拟环境
- ☐ 安装所有依赖包
- ☐ 下载并解压模型和数据集

2. 运行GUI程序

- ☐ 成功启动GUI界面
- ☐ 选择测试图片进行识别
- ☐ 观察识别结果

3. 运行摄像头/视频检测

- ☐ 启动实时检测程序
- ☐ 观察实时表情识别效果

6.2 进阶任务（选做）

1. 自定义图片测试

- ☐ 准备自己的表情图片
- ☐ 放入 `input/test/` 目录
- ☐ 运行识别并分析结果

2. 修改模型参数观察效果

- ☐ 查看 `src/model.py` 中的模型定义
- ☐ 尝试修改卷积核数量或全连接层大小
- ☐ 重新训练并比较效果

6.3 思考题

1. 在物联网边缘计算场景中，为什么模型使用48×48的小尺寸输入？这对嵌入式设备有什么意义？
 - 提示：考虑边缘设备的计算资源和实时性要求
2. BlazeFace模型仅约500KB，为什么它适合在物联网终端设备上部署？请从模型大小、推理速度、精度三个方面分析。
3. 本项目使用摄像头作为感知设备，请思考物联网系统中还有哪些传感器可以与表情识别结合应用？
4. 如果要将本系统部署到树莓派或嵌入式开发板上，需要考虑哪些优化措施？
 - 提示：模型量化、TensorFlow Lite、剪枝等技术
5. 请设计一个基于表情识别的物联网应用系统，画出系统架构图（包含感知层、网络层、应用层）。

七、常见问题解答

Q1: 运行时提示"ModuleNotFoundError"

原因：未激活虚拟环境或依赖未安装完整

解决：

```
conda activate FER
pip install -r requirements.txt
```

Q2: GPU版本TensorFlow无法使用GPU

原因：CUDA/cuDNN版本不匹配

解决：

- 确保安装了正确版本的CUDA (10.1) 和 cuDNN (7.6.5)
- 检查显卡驱动是否最新
- 如果仍无法使用，可以改用CPU版本

Q3: 摄像头打不开

原因：摄像头被其他程序占用或摄像头索引错误

解决：

- 关闭其他使用摄像头的程序
- 尝试不同的摄像头索引：`python recognition_camera.py --source 0` 或 `--source 1`

Q4: 人脸检测不到

原因：人脸太小、角度过大或光照条件差

解决：

- 确保正对摄像头
- 保持适当距离（不要太远）
- 确保光照充足

Q5: 识别结果不准确

原因：模型训练数据与实际场景差异

解决：

- 确保表情明显、夸张
- 尝试不同的表情角度
- 了解模型的局限性（训练数据特点）

Q6: 运行摄像头/视频检测时报 `cv2.imshow` 错误

错误信息:

```
cv2.error: OpenCV(4.5.1) ... error: (-2:Unspecified error) The function is not implemented.  
Rebuild the library with windows, GTK+ 2.x or Cocoa support.
```

原因: 安装了 `opencv-python-headless` 版本, 该版本不包含GUI功能 (`cv2.imshow`、`cv2.waitKey` 等)

解决:

```
# 1. 卸载所有OpenCV相关包  
pip uninstall opencv-python opencv-python-headless opencv-contrib-python -y  
  
# 2. 安装带GUI支持的版本  
pip install opencv-python==4.5.5.64  
  
# 3. 验证安装  
python -c "import cv2; print(cv2.__version__)"
```

说明: `requirements.txt` 中指定的 `opencv-python==3.4.4.19` 版本已不可用, 建议使用 `4.5.5.64` 或更高版本。

八、参考资料

1. 论文: *A Compact Deep Learning Model for Robust Facial Expression Recognition*
2. FER2013数据集: <https://www.kaggle.com/c/challenges-in-representation-learning-facial-expression-recognition-challenge>
3. BlazeFace: <https://github.com/tensorflow/tfjs-models/tree/master/blazeface>
4. TensorFlow官方文档: <https://www.tensorflow.org/>